

LAB 211 Assignment

Type:
Code:
LOC:
Slot(s):

Long Assignment
J1.L.P0038
300
N/A

Title

Car insurance Management

Background

Write a program to manage car insurance. The program implements the terminology of the Object-Oriented Programming (OOP) paradigm. OOP is one of the best ways to design software programs.

In this assignment, we will use the **Map**<Key, Value> structure to store a collection of objects. The Key is the identity of the object. Value is the object. The Map structure is commonly used to manipulate data efficiently in data management programs.

Program Specifications

Build the CarInsurance project with the menu as follows:

1. Add car information
2. Find a car
3. Update a car
4. Delete a car
5. Add an insurance statement
6. List of insurance statements
7. Report on uninsured cars
8. Save data
9. Load data
10. Quit

Each menu selection calls an appropriate function to execute the selected menu item to solve its meaning that serve the needs of the user. Your program must display a menu after each action and wait for the user to choose another option until the user chooses to quit the program.

Features:

Function 1. Add a car information - 30 LOC

- Request to enter information of a car: license plate, car owner, car brand, the value of the vehicle, Registration Date, registration place, and vehicle type.
- The Data needs to be verified with the following conditions:
 - All fields are not allowed null.
 - The “license plate” field must be unique.
 - The length of the “car owner” field must be from 2 to 35 characters.
 - The “Registration Date” field must be a valid date.
 - The “the value of the vehicle” field must be over 999.
 - The “vehicle type” must be one of the following: 5, 7, or 9 (*equivalent to 5 seats, 7 seats or 9 seats*).
- Add the vehicle information to the collection of cars.
- Ask to continue adding new vehicle information or go back to the main menu.

Function 2. Find a car – 30 LOC

- Require inputting the “license plate”.
- If the car does not exist, the message “Unregistered vehicle” will be displayed. Otherwise, display its info.

Function 3. Update a car – 30 LOC

- Require inputting the “license plate”.
- If the car does not exist, the message “Unregistered vehicle” will be displayed. Otherwise, the user can edit all vehicle information except the license plate.
- Show the result of the update: success or failure.

Function 4. Delete a car – 30 LOC

- Require inputting the “license plate”.
- If the car does not exist, the message “Unregistered vehicle” will be displayed. Otherwise, the user can delete the car information.
- Must show the confirmation message before deleting.
- The car information cannot be deleted if it is already registered in insurance
- Show the result of the deletion: success or failure

Function 5. Add an insurance statement – 30 LOC

- An insurance statement includes the insurance id, the established date, license plate, customer name, insurance period, and insurance fees.
- The Data needs to be verified with the following conditions:
 - All fields are not allowed null.
 - The “insurance id” fields must be unique.
 - The “established date” field must be a valid date.
 - The “insurance period” must be one of the following: 12, 24 or 36.
 - The “insurance fees” must be a positive integer value and calculate by formular:
 - 25% of “the value of the vehicle” field if insurance period is 12;
 - 20% of “the value of the vehicle” field multiply with 2 if insurance period is 24;
 - and 15% of “the value of the vehicle” field multiply with 3 if insurance period is 36
- Add the insurance statement to the collection of insurance statements.
- Ask to continue adding a new insurance statement or go back to the main menu.

Function 6. List of insurance statements – 30 LOC

- Year value is required to display the insurance statements.
- Required to enter the field which the user wants to sort (insurance id, or established date, or license plate, or car insurance period)
- Select sort type to display the list by ASC or DESC on which field was chosen by the user
- List of insurance statements for that year as below if applicable.

Report : INSURANCE STATEMENTS
From : 01/01/2023 To: 12/31/2023

Sorted by: Established Date
Sort type : ASC

No.	Insurance Id	Established Date	License plate	Customer	Insurance period	Insurance fees
1	0006	01/17/2023	50H3-123.45	John Adams	12	\$ 6000
2	0007	01/28/2023	51H1-888.88	Jamie Oliver	12	\$ 9000
3	0008	02/14/2023	54H1-567.89	Will Smith	24	\$ 14,400
4	0009	03/11/2023	50H1-222.22	Alan Nguyen	36	\$ 12,600

Function 7. Report uninsured cars – 30 LOC

- Required to enter the field which the user wants to sort (license plate, or car owner, or registration date, or vehicle type) and the sort type (ASC, DESC).
- Sort and display the collection of car information as below.

Report: UNINSURED CARS

Sorted by : Vehicle type
Sort type : DESC

No.	License plate	Registration Date	Vehicle Owner	Brand	Vehicle type	Value
1	51H2-370.11	11/15/2022	Tuan Nguyen	Ford	7	\$ 41000
2	50H3-612.00	02/26/2023	John Smith	Mitsubishi	7	\$ 38500
3	51H5-011.12	01/30/2023	Martin Vu	Honda	5	\$ 21800
4	50H1-412.33	10/01/2022	Hoa Tran	Toyota	5	\$ 16,500

Function 8. Save data – 30 LOC

- Save the collection of vehicles to the binary file **carInfo.dat**.
- Save the collection of insurance statements to the binary file **insurances.dat**.

Function 9. Load data – 30 LOC

- Load the collection of vehicles from the binary file **carInfo.dat**.
- Load the collection of insurance statements from the binary file **insurances.dat**.

Function 10. Quit – 30 LOC

- Exit the program.
- Must show the confirmation message before exiting.
- Must save data to files if data changes.

The above specifications are basic information only; you have to perform a requirements analysis on step and build the application according to the actual requirements.

The lecturer will only explain the requirement once in the first slot of the assignment.